



ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ

**Τμήμα Ηλεκτρολόγων Μηχανικών
και Μηχανικών Υπολογιστών**

Διπλωματική Εργασία

Δοκιμαστική Έκδοση

Όνομα Φοιτητή	Βαπόρης Δημήτριος
ΑΕΜ	10625
Επιβλέπων Καθηγητής	Ντελόπουλος Αναστάσιος

Εαρινό Εξάμηνο 2026

Περιεχόμενα

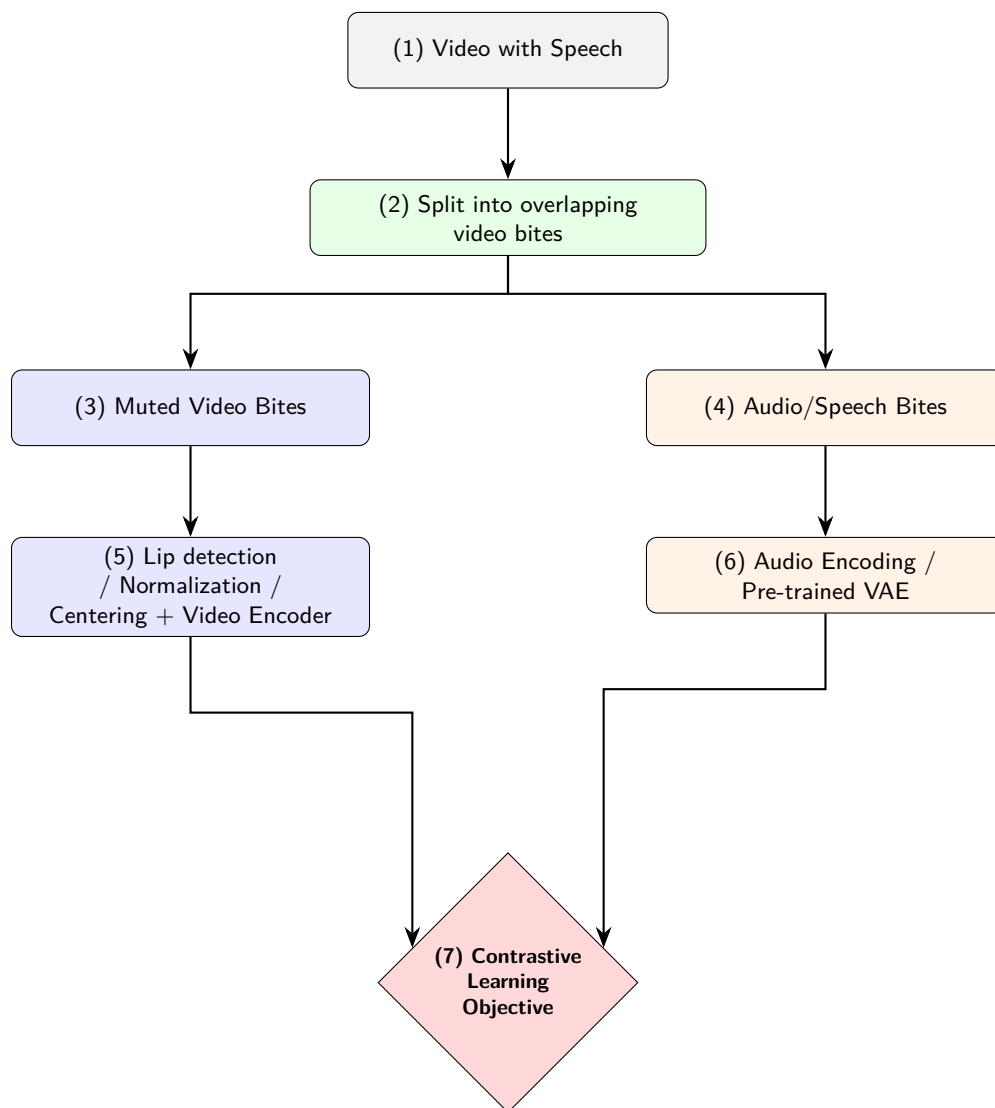
1	Εισαγωγή	2
2	Proposed Method	2
2.1	Contrastive Learning	2
2.2	Generation by Diffusion	3
3	Testing of singular features of the proposed method	4
3.1	Testing the video encoder and the audio encoder-decoder	5
3.2	Testing audio and video alignment	9
3.3	Contrastive Learning solely on audio files	10
3.4	Contrastive learning solely on video files	12
4	Original Time Domain and MFCCs	12
4.1	Original Time Domain signals	12
4.2	Mel-frequency cepstral coefficients	13

1 Εισαγωγή

2 Proposed Method

A combination of Contrastive Learning methods and a diffusion generation mechanism is proposed, known as a Latent Diffusion Model (LDM). The following schematics aim to provide a visual explanation of the process.

2.1 Contrastive Learning



Σχήμα 1: The Contrastive Learning part of the proposed method

The method proposed is very similar to the one proposed in Diff-Foley [1]. More specifically, each step of the above schematic is explained below:

(1): The process begins with data from a video with speech.

(2): The video is divided into overlapping video bites (overlap % is a tunable parameter) of length capable of depicting a specific sound (syllable?) made during speech (around 0.1-0.3 seconds, also a tunable parameter).

(3), (4): The data is split into muted video bites and audio/speech bites.

(5): The data of the muted video bites is pre-processed using a combination of the following techniques:

- Face/Lip Detection: To avoid meaningless information connected to speech (i.e. background color, clothes of speaker...) a face/lip detection mechanism is used to reduce the size of data passed to next stages.
- Normalization/Centering: Prevent the model from confusing head tilt/shake as an important part of speech (could be redundant based on architecture used later).
- Possible conversion to Grayscale instead of RGB to further reduce the data incorporated.
- Masking: Two forms of masking are possible, temporal and spatial. Temporal masking involves adding noise to (or completely covering) a relatively small number of frames of the video bite in their entirety, whereas spatial masking constitutes of adding noise to specific areas of the video bite (i.e. near the left part of the lip) for the entire video bite (or, again, covering/removing them entirely).

The final processed data is passed into the Video Encoder (i.e. VideoMAE [2], works with a high masking percentage).

(6): The audio is also processed and encoded, using the Mel-spectrogram. A Pre-trained Audio Variational Autoencoder (VAE) is used, since it is trained to both encode and decode to and from the resulting latent space, while retaining phase information (crucial for correct reconstruction and alignment of lip movements and sounds generated). Examples of architectures to be considered: HTS-AT [3] or the VAE used in AudioLDM [4]. Due to issues with the dependencies of the video encoder alongside the audio encoder, EnCodec from Meta was chosen [8].

(7): Lastly, the Contrastive Learning Objective is applied, by the result of which the fine-tuning of the encoders is driven. The Contrastive Learning Objective is very similar to the one presented in Diff-Foley [1], containing a Semantic Contrast and a Temporal Contrast. The Semantic Contrast aims to maximize the similarity score of video-audio bite pairs coming from the same video while minimizing the score of video-audio bite pairs coming from different ones (if not using many different videos with different speakers, then totally different parts of the video, maybe with a completely different context/emotion/tone of the speaker). The Temporal Contrast aims to maximize the similarity of video-audio bite pairs that come from the same time segment of the video, whereas it minimizes the similarity score of pairs coming from different time segments, making precise timing crucial (this is why an overlap is also proposed, to enhance learning of precise timing).

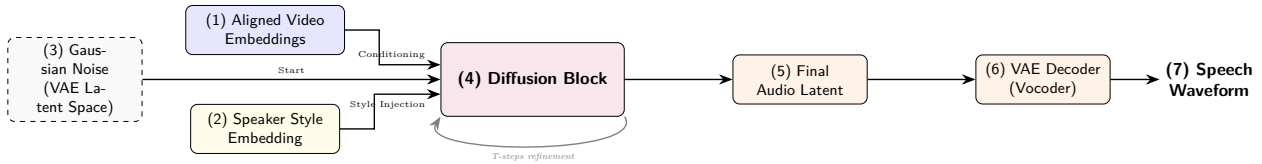
2.2 Generation by Diffusion

Having completed the training of the Contrastive Learning part, the outputs of the Video Encoder as well as the VAE are aligned in a common latent space, meaning that muted video bites are projected via the Video Encoder to a point in the latent space that resides close to the projection of the missing sound/speech, after being passed through the VAE. This fact is taken advantage of in order to drive or rather condition the diffusion generation process, which can be visualized below.

Below, a brief explanation of the modules above is included.

(1): As discussed previously, having completed the Contrastive Learning part of the method creates an aligned latent space, meaning that the embedding resulting from the video bite for which speech is being generated can be used as conditioning for the diffusion process (maybe also neighboring video bites?).

(2): An optional part of the process that may help alleviate robotic sound and generalize the process is the usage of the speaker's style embedding as a secondary conditioning mechanism (a



Σχήμα 2: The Generation by Diffusion part of the proposed method

“style injection”) for the diffusion. Using pre-trained Speaker Encoders (like WavLM [5]) that encapsulate information like the speaker’s tonality from a reference speech clip, the result of the diffusion process may be steered so as to attempt to “match” the desired speaker tone.

(3): The first step of the diffusion process described below is a Gaussian Noise vector in the latent space of the Audio VAE. The diffusion process aims to “denoise” the initial vector and finally create a vector in the latent space that may then be decoded as a speech segment valid for the video bite at hand, conditioned (driven) by the aligned video embeddings and, optionally, the speaker’s style. Importantly, during sampling, a scaled version of noise is added in each step of the diffusion process except for the last one (the one that results in the outcome), as shown in DDPM [6].

(4): The Diffusion Block is the cornerstone of generation for the proposed method. During training, the Diffusion Block receives an Audio Latent as well as the corresponding Video Latent created from the VAE and the Video Encoder respectively. Since the objective is speech generation, Gaussian Noise is added to the Audio Latent. The objective of the Diffusion Block during training is, given the noisy Audio Latent and the Video Embedding, to predict how much noise was added to the Audio Latent. The loss function stems from the error in the actual noise added versus the predicted noise. Having completed training, during inference (generation), random noise is sampled for the first step (as discussed in 3). The Video Embedding serves as a conditioning mechanism, driving the process towards a less noisy Audio Latent that, after enough steps (also a tunable parameter), resides close in the Aligned Latent Space to the Video Embedding (hopefully...). The optional Speaker Style Injection acts as a constant bias that ensures the resulting denoised latent maps to a speech segment with the same tonal characteristics of the target speaker.

(5): After T steps of refinement in the inference process, a final Audio Latent is produced.

(6): This final Audio Latent is then passed into the VAE Decoder.

(7): The result of the VAE Decoder is now a speech waveform with length equal to the length of the initial video bite.

Following this process, a speech waveform is produced for every video bite of the desired video. However, simply adding these bites on top of one another to create the final speech segment would create discontinuities in the sound, especially at the end and the beginning of each bite. Thus, a need for combination of the speech waveforms is presented, which is the main reason overlapping video bites are created from the beginning. The overlapping generated audio bites are combined using e.g. Hanning windows to cross-fade the individual segments and create the final desired speech segment.

This proposed method is largely subject to change, upon discussion with the supervisor.

3 Testing of singular features of the proposed method

Before testing the method in its entirety, tests were conducted with each module of the method separately. This led to a more solid understanding of the process as well as every singular module. The process included testing the audio encoding and decoding of the chosen audio VAE (EnCodec [8]), testing the encoding capabilities of the video encoder chosen (VideoMAE v2 [7]), testing Contrastive Learning solely on audio segments from speech (not in contrast to the videos they originated from) using viable data augmentation techniques which will be discussed below, as well as similarly testing

Contrastive Learning solely on the video segments while augmenting the data properly.

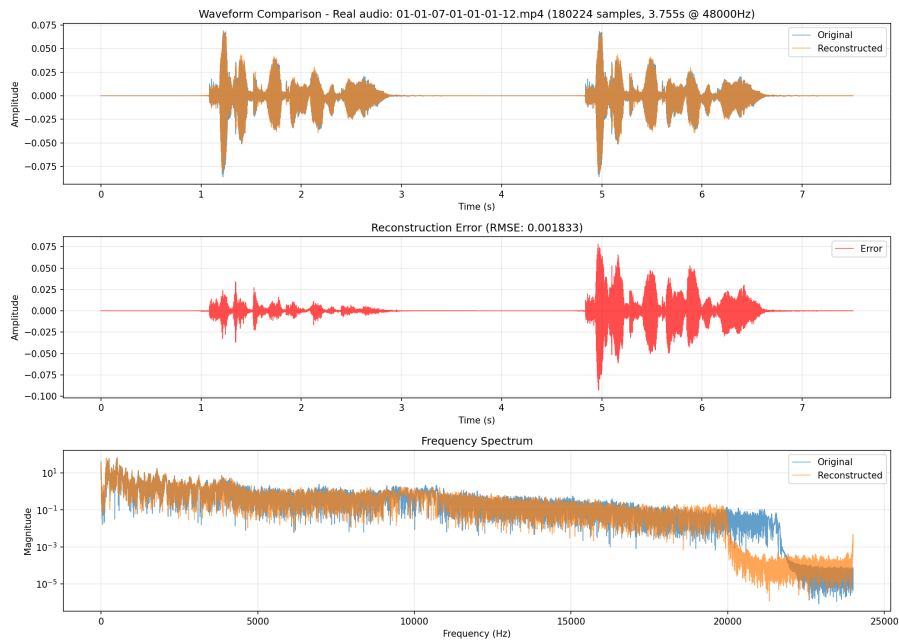
Some key observations crucial to the understanding of the results below are showcased in this section:

- The dataset chosen for this purpose is RAVDESS [9], which contains both videos and pure audio of actors speaking or singing specific phrases. The videos are .mp4 files with resolution 1280×720, a framerate of 29.97 fps and stereo sound (which is ignored since pure audio files are also contained in the dataset). The audio files are .wav.
- The video encoder VideoMAE [2] produces a 768-dimensional vector embedding per 16-frame video segment. At 29.97 fps, this results in video bites with length around 0.534 seconds. Importantly, the video encoder does not provide a meaningful decoding from the embedding space, meaning the opposite reconstruction (video of speech from audio) is impossible with the current setup.
- The audio VAE [8] receives stereo 48kHz audio (the audio files are actually mono because they are created from the mp4 files of the videos for perfect alignment, but the samples are cloned to use the stereo model), which for a duration of about 0.534s amounts to 25626 samples, returning a matrix that leads to a flattened vector of dimension 1296 (a compression of a factor close to 20). This number is the result of this operation: $25626 \text{ (total samples in audio bite)} / 320 \text{ samples/frame} \approx 80 \text{ frames (rounding up to 81)}$, with each frame encoded in 16 dimensions, resulting in a 16×81 array or a flattened vector of dimension 1296.
- For the contrastive learning, it is suggested that a fully connected layer is used to project from the initial 1296-dimension vector to an intermediate dimensional space and then to the latent space, which will be the common embedding space for audio and video.

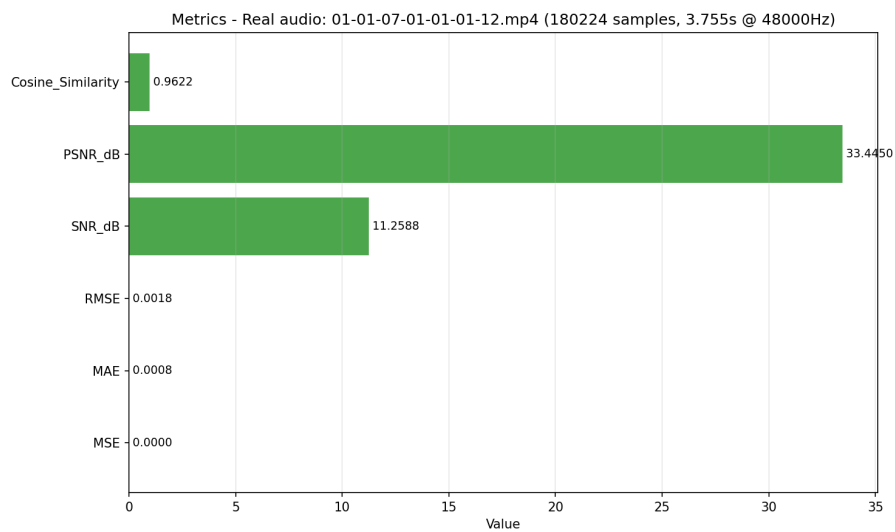
3.1 Testing the video encoder and the audio encoder-decoder

Since the video encoder [2] does not include a meaningful decoder from the 768-dimensional space it maps videos to, only the ability to encode video segments (either generated or from the dataset) was tested in this part. This introductory test was meant as a way to interpret the resulting dimensional space from the encoding, as well as familiarize with the usage of the video encoding process.

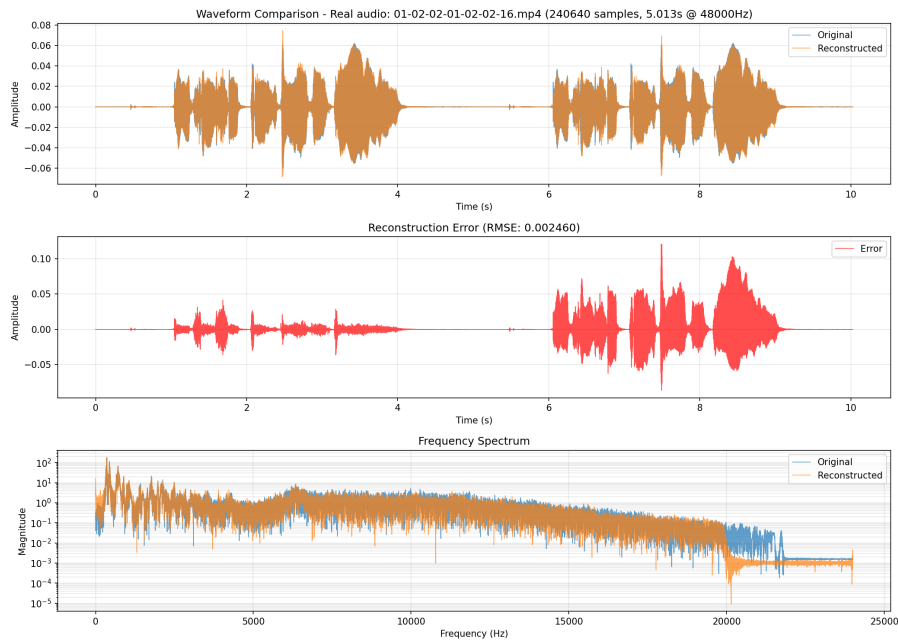
As part of this first stage of testing, a handful of audio encoding-decoding tests were conducted. To start, entire audio files were encoded and decoded with the EnCodec [8] quite successfully. Some indicative results are presented below:



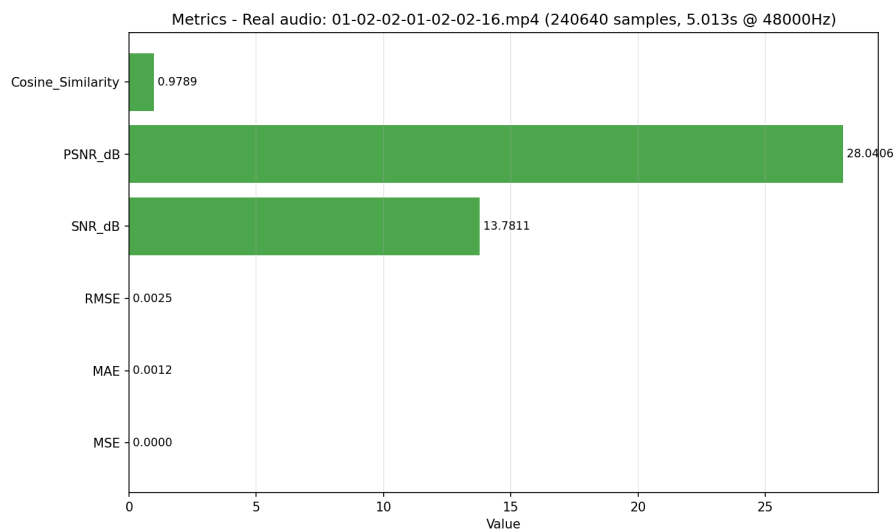
Σχήμα 3: Waveform comparison, reconstruction error and frequency spectrum of 1st test audio file



Σχήμα 4: Metrics of reconstruction of 1st test audio file

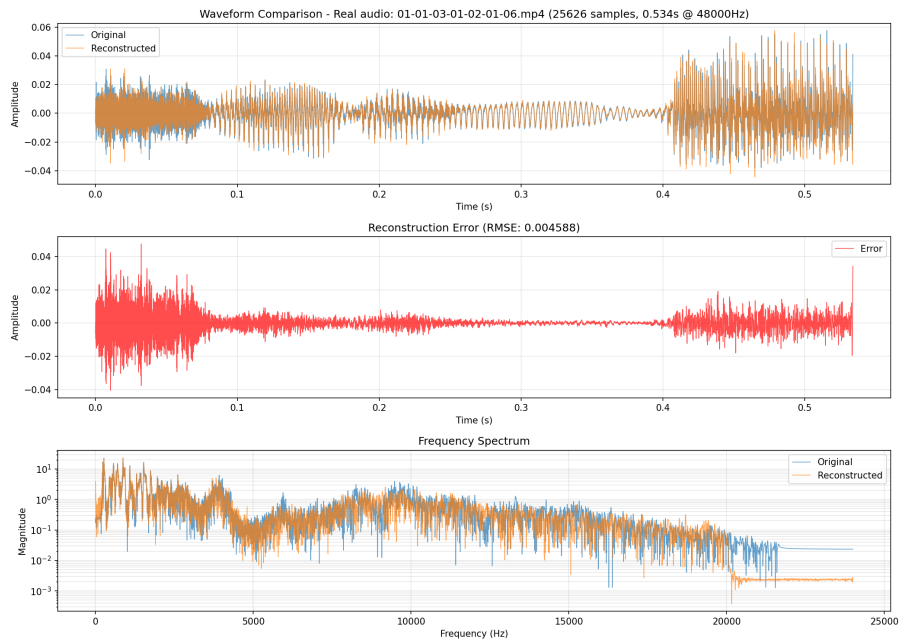


Σχήμα 5: Waveform comparison, reconstruction error and frequency spectrum of 2nd test audio file

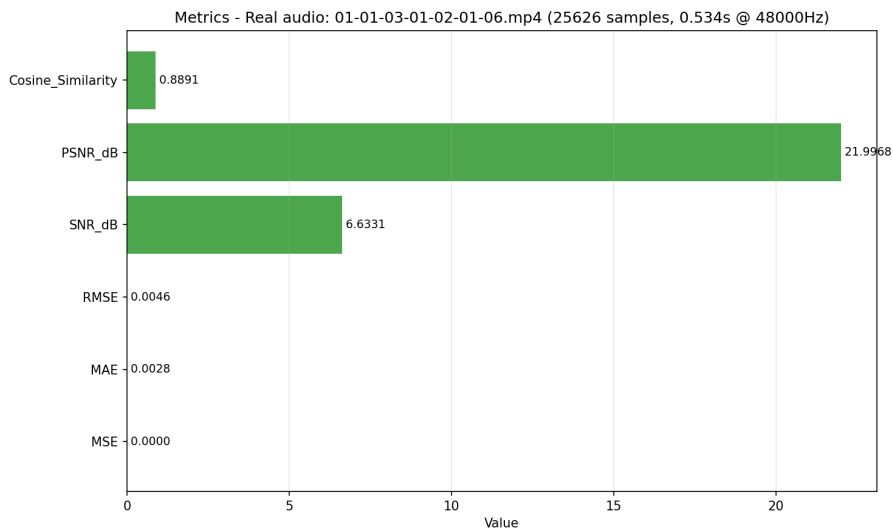


Σχήμα 6: Metrics of reconstruction of 2nd test audio file

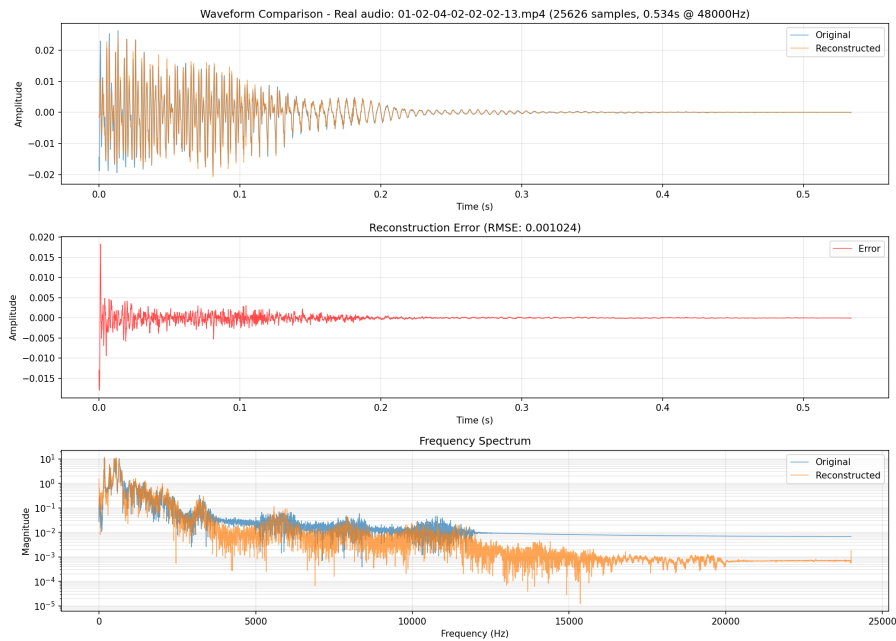
The encoder showcases a very accurate reconstruction ability, with very low errors and high cosine similarity and SNR and PSNR metrics. This process was also tested for random audio segments that correspond to the 16 frame windows that are created for the video encoder (a time length of around 0.534s).



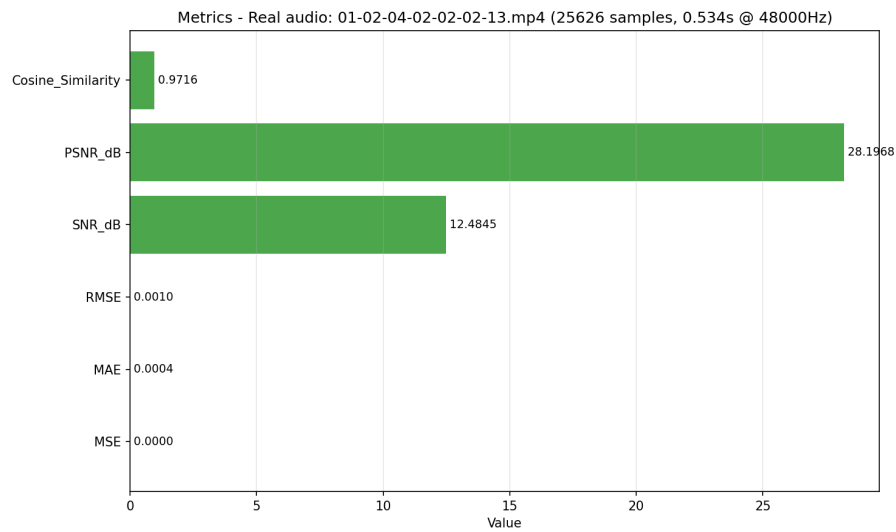
Σχήμα 7: Waveform comparison, reconstruction error and frequency spectrum of 1st segmented test audio file



Σχήμα 8: Metrics of reconstruction of 1st segmented test audio file



Σχήμα 9: Waveform comparison, reconstruction error and frequency spectrum of 2nd segmented test audio file

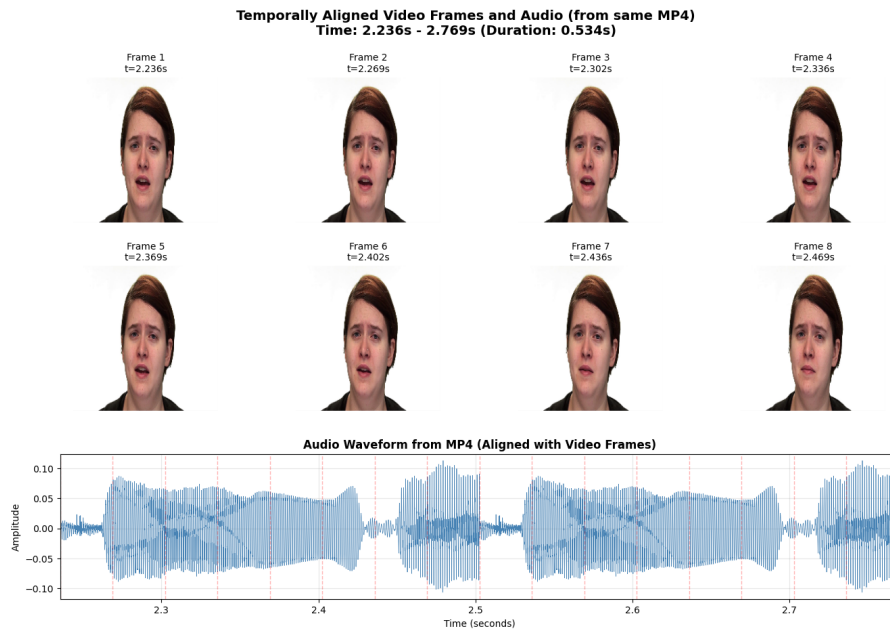


Σχήμα 10: Metrics of reconstruction of 2nd segmented test audio file

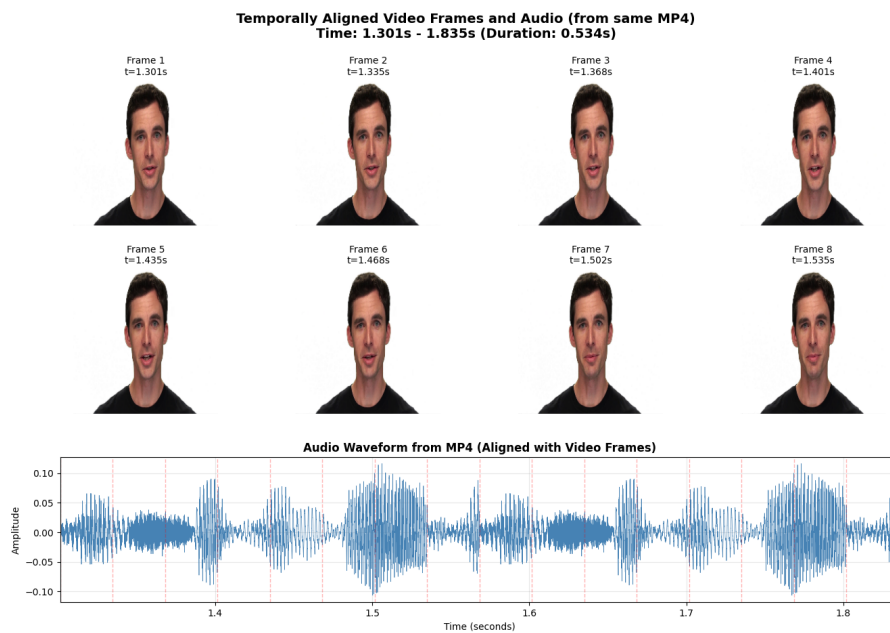
Even with this smaller audio length to work with, the encoder-decoder manages very pleasing results, with extremely high similarity scores and SNR's, while keeping reconstruction error metrics very low.

3.2 Testing audio and video alignment

In order to ensure proper temporal alignment of the video and audio bites (length of 16 frames, around 0.534s), a separate testing script was created. This script showcases the complete pipeline of reaching a video from the kaggle dataset, extracting 16 frames from it and ensuring that the resulting video and audio are properly aligned temporally. The results are shown below:



Σχήμα 11: Temporal alignment of 1st test video file



Σχήμα 12: Temporal alignment of 2nd test video file

Obviously the correctness of the results cannot be proven with the provided figures, but they serve as an illustration of the subjective correctness the alignment of the files themselves display.

3.3 Contrastive Learning solely on audio files

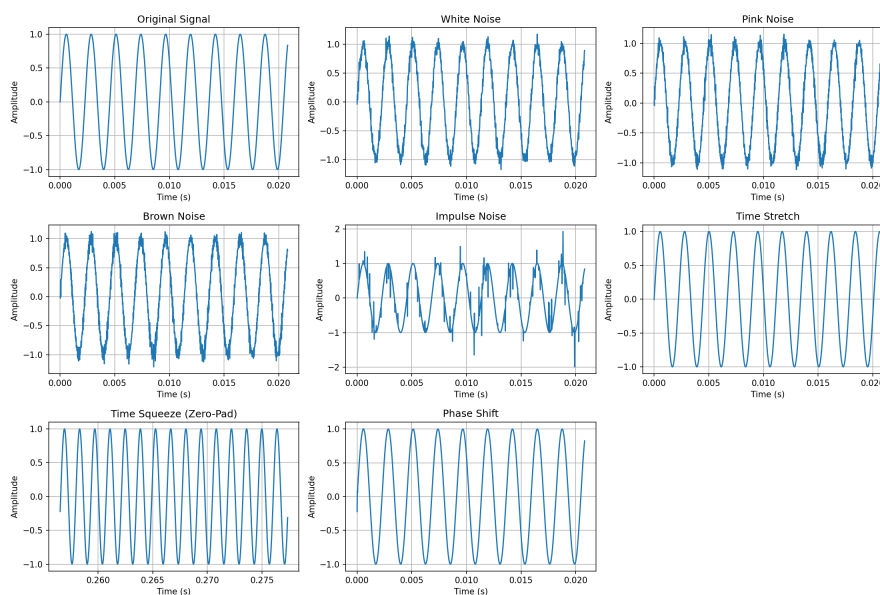
In order to test the applicability of multimodal contrastive learning and whether the process converges or not, it was deemed necessary that single mode contrastive learning for both audio and muted video (see next section) be tested first. For this purpose, audio files that stem from the .mp4 files of the dataset are used as the training, validation and testing data with a proper split percentage. It is also ensured that the audio bites used exhibit an overlap percentage of around 50% and that the length is approximately equal to the length of a 16 frame segment of the video.

For the purposes of this study, a few methods of data augmentation for audio files were implemented. These include:

- Addition of white noise: equal power among all frequencies
- Addition of pink noise: power follows a $1/f$ distribution.
- Addition of brown noise: power follows a $1/f^2$ distribution.
- Addition of impulse noise: adds random but controlled spike noise.
- Time stretching while maintaining the same output length.
- Time squeezing with zero-padding to ensure same output length.
- Phase Shift in the frequency domain.
- Manifold Mixup: mixing two features in the embedding space as a linear combination where the linear combination parameter λ follows a beta distribution. This ensures that the model learns to predict less confidently on interpolations of hidden representations, creating smoother decision boundaries in cases of classification, or, in our case, allowing for interpolation of sounds. [10]

Since the encoding and decoding mechanism is very subtle and sensitive to change, the VAE remains frozen during this stage. Only a feature extraction head is trained, which maps the 1296-dimensions vector to a lower dimension (in which the contrastive learning objective is applied). The contrastive learning objective is similar to the one proposed earlier, consisting of a semantic contrast (do the 2 audio files originate from the same video, do they come from the same speaker, does the speaker showcase the same emotion...) and a temporal contrast (do the two audio files come from around the same time in the video, are the contents of the two files similar) component.

Initially, a test script was implemented to showcase that all of the above processes, including the data augmentation methods, are operational. These are some indicative results of the data augmentation processes:



Σχήμα 13: Results of data augmentation methods

Out of the methods presented above, phase shift, time stretch and time squeeze are quite computationally expensive, so for an initial test they may be skipped. The contrastive learning script for audio files is ready, however the computational cost is quite high and therefore it was deemed necessary that the Institutional Cluster be used. This requires an additional period of learning and setup on my behalf, so progress has been a bit slower than expected.

3.4 Contrastive learning solely on video files

Currently under consideration. I will employ a method similar to the audio case. In this case, however, there is the extra freedom of being able to fine-tune the video encoder, since decoding from the video embedding is impossible in this current setup. In addition to training the feature extraction head, some layers of the Video Encoder may be unfrozen so as to improve training.

4 Original Time Domain and MFCCs

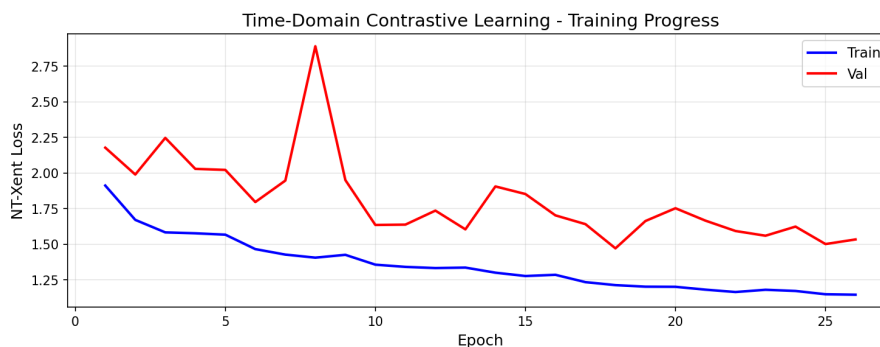
Following the disappointing results of the training of the previous method, two different approaches were tested for the Contrastive Learning method of speech segments. A method similar to SimCLR [12] was applied for inputs being samples both in the Original Time Domain as well as MFCCs (Mel-frequency cepstral coefficients).

4.1 Original Time Domain signals

Initially, original time domain signals were used as samples for the contrastive learning objective. Using non-overlapping speech segments with length 0.5s, a “speech chunk index” was built and enhanced with the results of various methods of data augmentation to these speech segments: white, pink and brown noise, phase shift as well as time stretch. Each speech segment was augmented exactly once with one of the previous techniques.

The model trained included 4 1-dimensional convolutional layers with batch normalization, a ReLU activation function and pooling (max pooling for the first 3 layers and average pooling for the last layer). The resulting dimension after the convolutional layers was 512. The projection head included a linear layer with batch normalization, a ReLU activation function and dropout, leading to a hidden dimension of 512. Lastly, another linear layer led to the final 128-dimensional space.

The model was trained with the NT-Xent loss (the same one used in SimCLR [12]), an Adam optimizer and a learning rate scheduler to ensure smoother convergence. Positive pairs only included the pairs corresponding to the original time domain sample and the result of its augmentation. These are the results of the training:



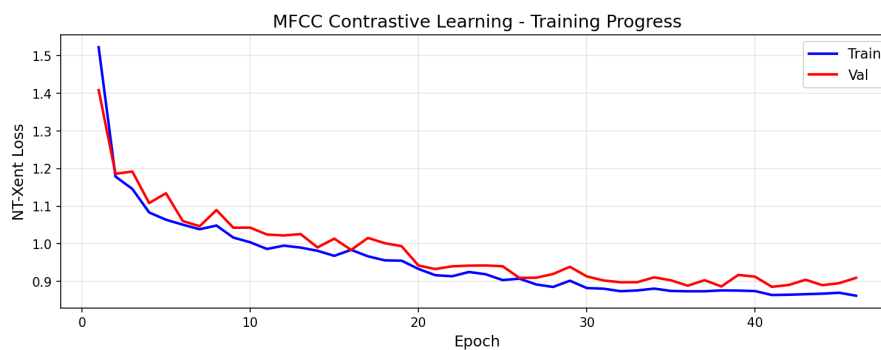
Σχήμα 14: Results of training in the original time domain

The results indicate learning but inability to properly adapt to the validation set, since the validation curve seems to plateau quite early. Training is also not smooth since there is a spike in the validation loss during epoch 8.

4.2 Mel-frequency cepstral coefficients

As a follow-up to the previous method, the use of MFCCs was also tested, since they are widely used in speech recognition systems. Each speech segment is resampled using a frequency of 16kHz, resulting in 8000 samples for each audio chunk. Following the calculation of the MFCCs, these 8000 samples result in 13 MFCCs for each of 48 time frames, meaning the output is a tensor with shape [13,48].

Since the results of MFCC application are 2-dimensional, it is highly efficient to use the ResNet pre-trained model and finetune it, since it has been shown to learn various important patterns in 2-dimensional space. The same data augmentations techniques were used as in the previous approach. The projection head was also the same. ResNet-18 was chosen as the ResNet model to be used, with a minor tweak needed in the first convolutional layer of the model to accept 1-channel inputs. All other aspects of training were kept the same to enable a comparison between the methods. These are the results:



Σχήμα 15: Results of training in the original time domain

Overall, much better values of the loss function were achieved for both the training and the validation sets, while the convergence was also a lot smoother. After around 35 epochs, validation loss hit a plateau at values of around 0.9.

Αναφορές

- [1] S. Luo, C. Yan, C. Hu and Hang Zhao, “Diff-Foley: Synchronized Video-to-Audio Synthesis with Latent Diffusion Models”
- [2] Z. Tong, Y. Song, J. Wang, L. Wang , “VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training”
- [3] K. Chen, X. Du, B. Zhu, Z. Ma, T. Berg-Kirkpatrick, S. Dubnov, “HTS-AT: A HIERARCHICAL TOKEN-SEMANTIC AUDIO TRANSFORMER FOR SOUND CLASSIFICATION AND DETECTION”
- [4] H. Liu, Z. Chen, Y. Yuan, X. Mei, X. Liu, D. Mandic, W. Wang, M. D. Plumbley, “AudioLDM: Text-to-Audio Generation with Latent Diffusion Models”
- [5] S. Chen, C. Wang, Z. Chen, Y. Wu et. al, “WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing”

- [6] J. Ho, A. Jain, P. Abbeel, “Denoising Diffusion Probabilistic Models”
- [7] Wang, Limin and Huang, Bingkun and Zhao, Zhiyu and Tong, Zhan and He, Yinan and Wang, Yi and Wang, Yali and Qiao, Yu, “VideoMAE V2: Scaling Video Masked Autoencoders With Dual Masking”
- [8] A. Défossez, J. Copet, G. Synnaeve, Y. Adi, “High Fidelity Neural Audio Compression”
- [9] S. R. Livingstone, F. A. Russo, “The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English.”
- [10] V. Verma, A. Lamb, C. Beckham, A. Najafi, I. Mitliagkas, A. Courville, D. Lopez-Paz, Y. Bengio, “Manifold Mixup: Better Representations by Interpolating Hidden States”
- [11] D. Jiang, W. Li, M. Cao, W. Zou, X. Li, “Speech SimCLR: Combining Contrastive and Reconstruction Objective for Self-supervised Speech Representation Learning”
- [12] T. Chen, S. Kornblith, M. Norouzi, G. Hinton , “A Simple Framework for Contrastive Learning of Visual Representations”
- [13] A. Baevski, H. Zhou, A. Mohamed, M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations”